SAI THARUN A / 192321116

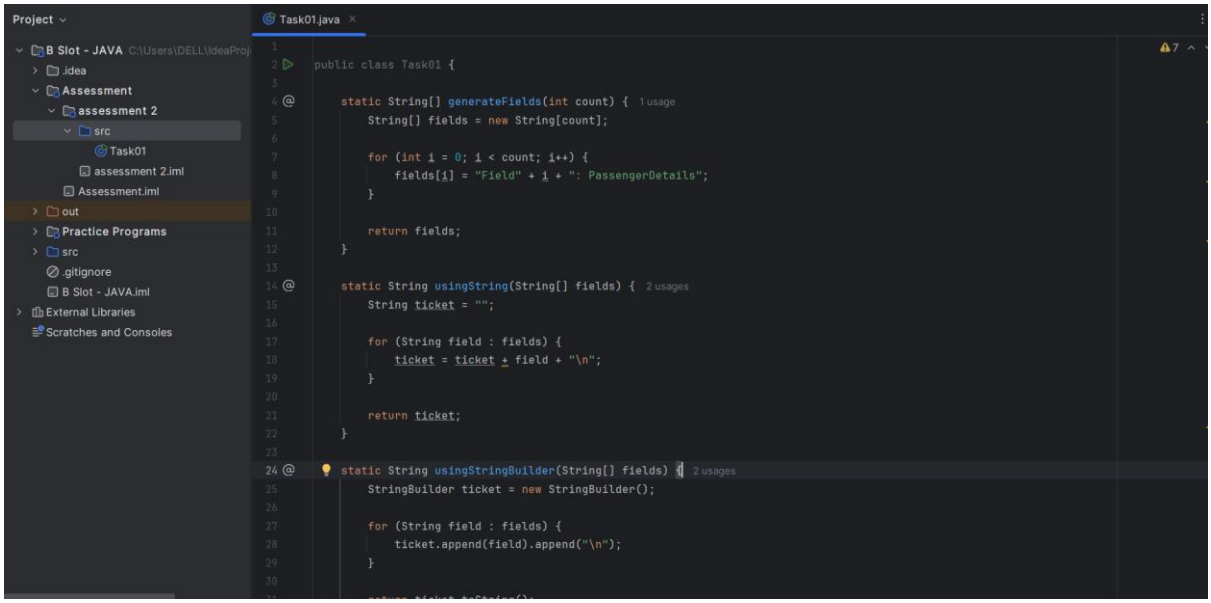
CSA0920 – JAVA PROGRAMMING

ASSIGNMENT - 02

Q1–Building Text Under Load: String, StringBuilder or StringBuffer?

Bloom's Level: K4– Analyse Problem Context– Railway e-Ticket Generation

A railway reservation system generates an e-ticket for every booking. Each e-ticket is built by joining many fields– PNR, passenger names, coach, berth, fare and journey details– into a single printable text block. The current code builds the ticket using the + operator inside a loop.



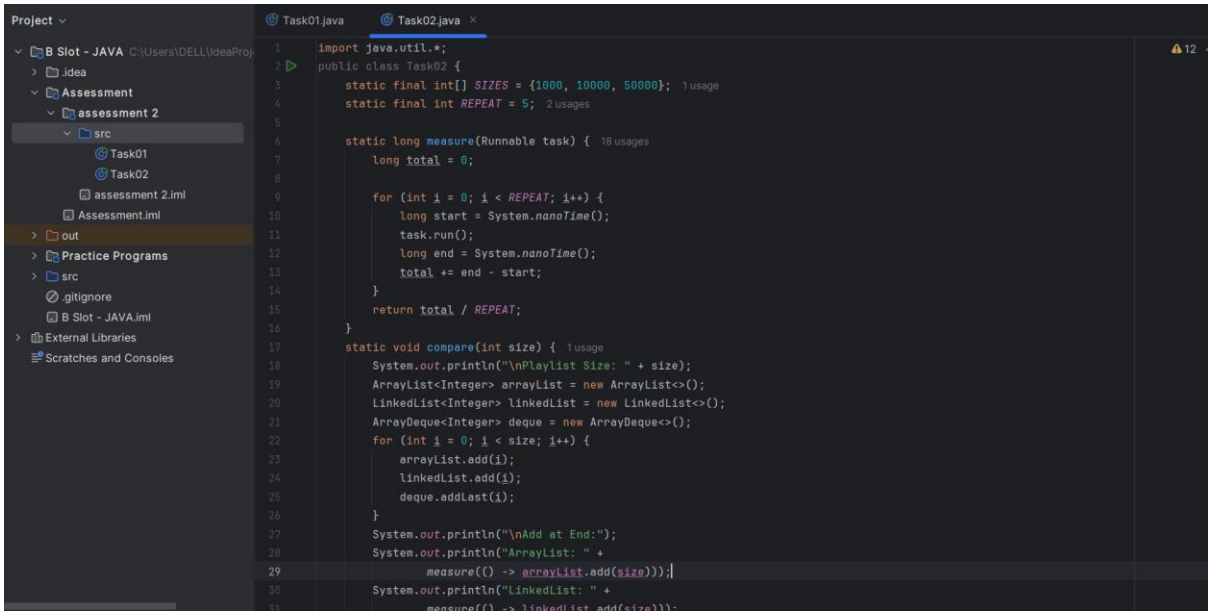


TASK 2

Q2–When Is ArrayList an Appropriate Engineering Choice?

Bloom's Level: K4– Analyse Problem Context– Music Playlist Editor

A music-streaming app lets users build playlists. Users add songs at the end, insert songs at chosen positions, remove songs and jump directly to the k-th song. The app currently stores every playlist in an ArrayList<Song>.



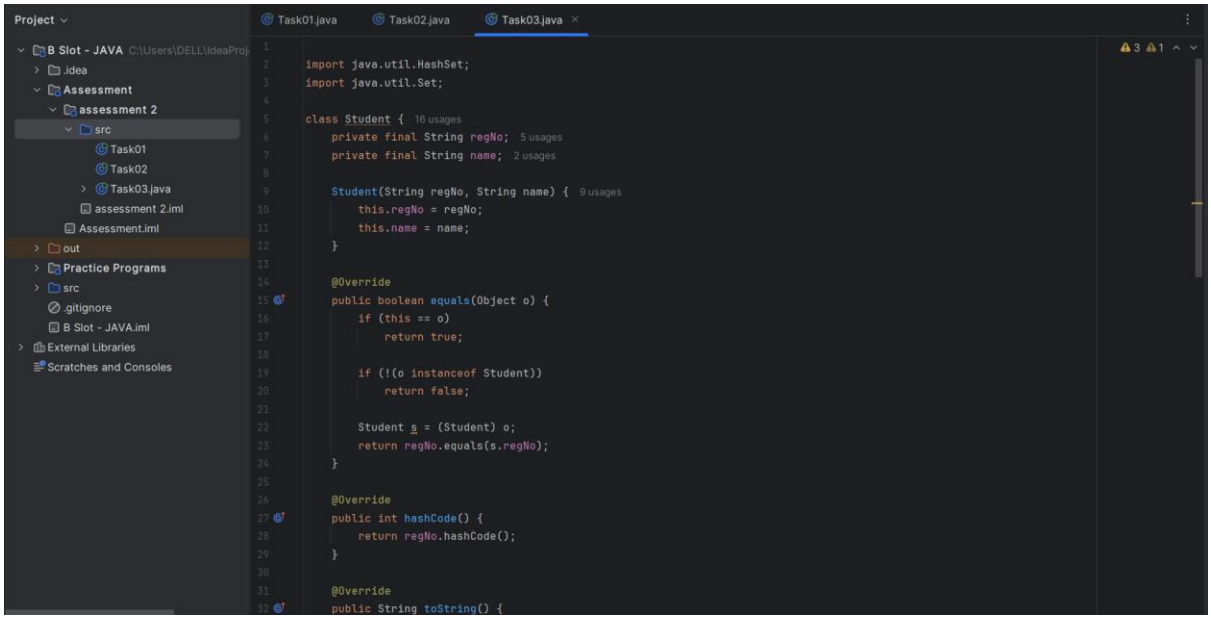


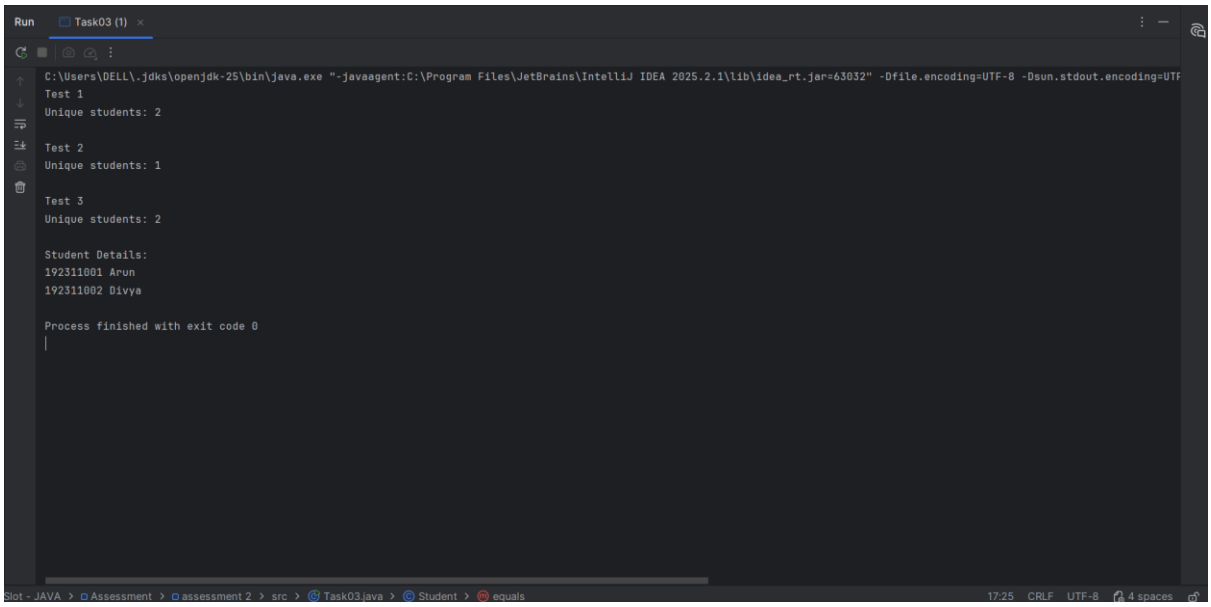
TASK 03

Q3–Engineer the hashCode–equals Contract

Bloom's Level: K4– Analyse Problem Context– Event Attendance De-duplication

A college fest records attendance by scanning student ID cards at several gates. The same student may be scanned more than once. The organisers store scans in a HashSet<Student> to keep only unique students.



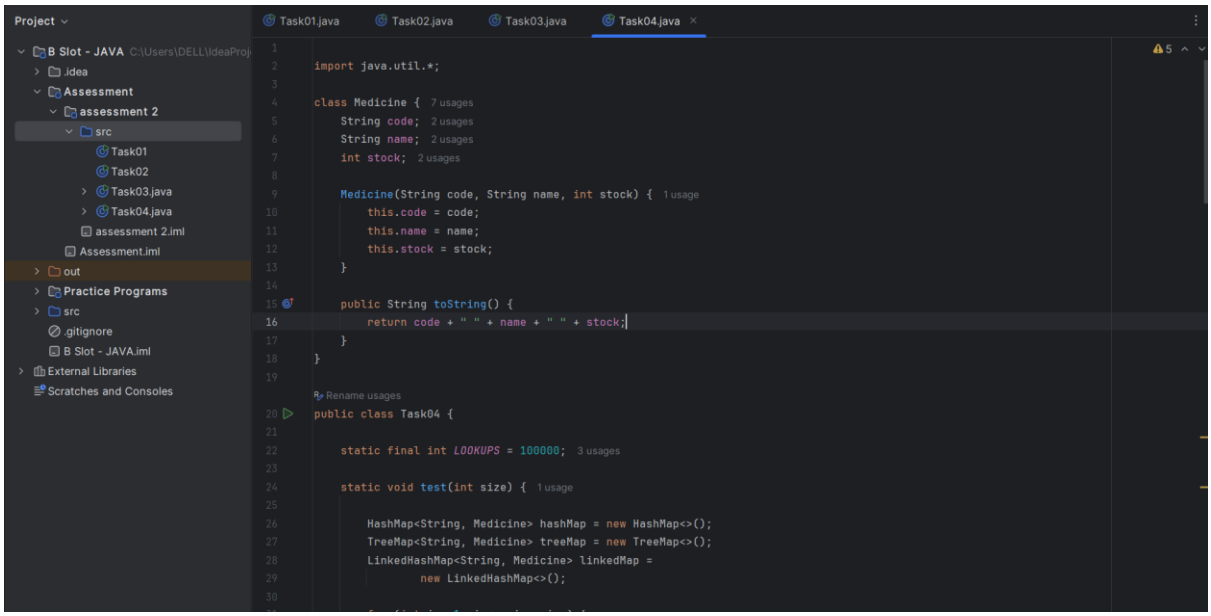


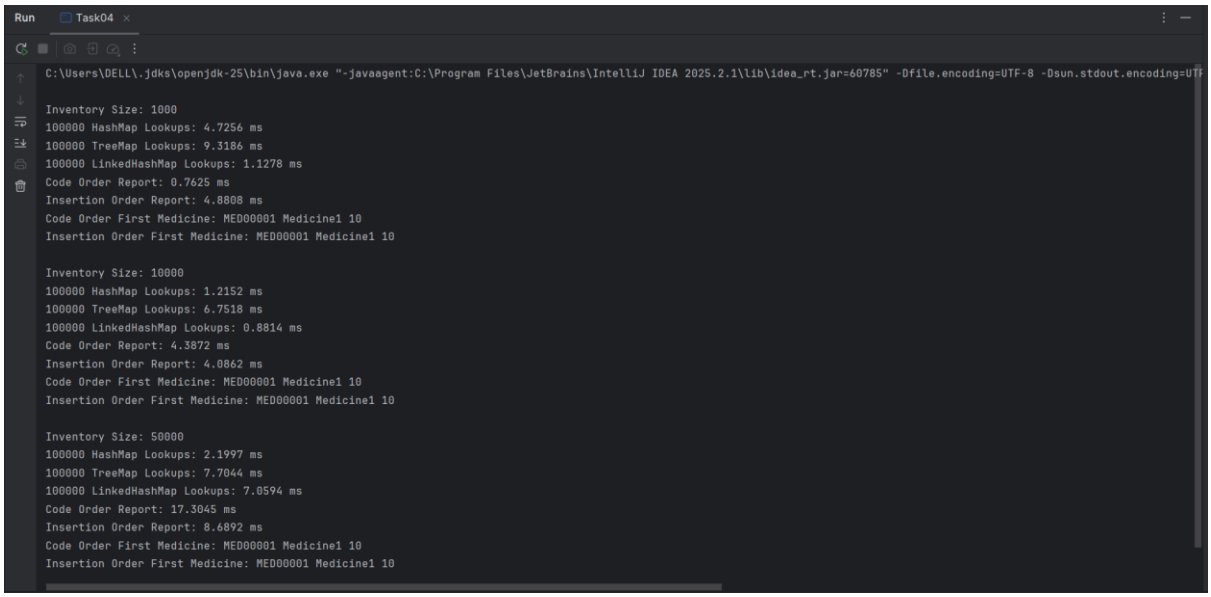
TASK 04

Q4–Choosing a Mapfor a Pharmacy Inventory Bloom's

Level: K4– Analyse Problem Context– Hospital Pharmacy Stock Lookup

A hospital pharmacy keeps stock for each medicine, identified by a medicine code. Pharmacists look up stock by code hundreds of times a day, and every evening the system prints a report of all medicines in code order and a list of medicines in the order they were added.



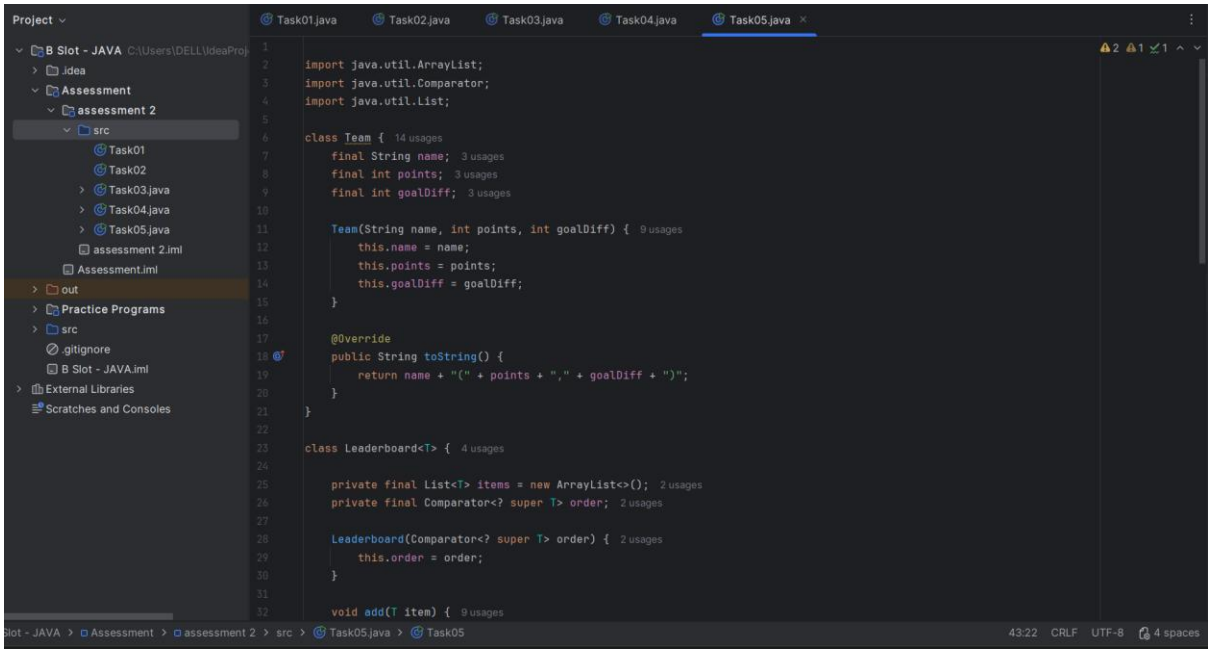


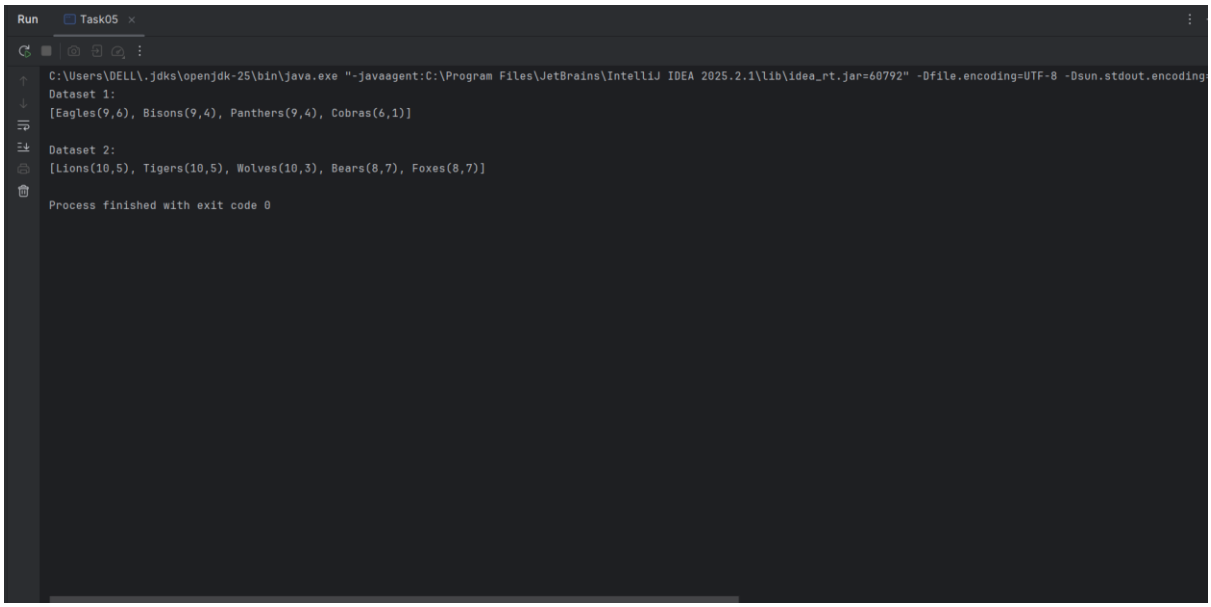
TASK 05

Q5–Derive a Generic Ranking Component from a Principle

Bloom's Level: K4– Analyse Problem Context– Inter-College Sports Leaderboard

A sports meet ranks teams by points (higher first). Ties are broken by goal difference (higher first), then by team name (alphabetical). The same ranking component will later be reused for ranking players, colleges and quiz teams.



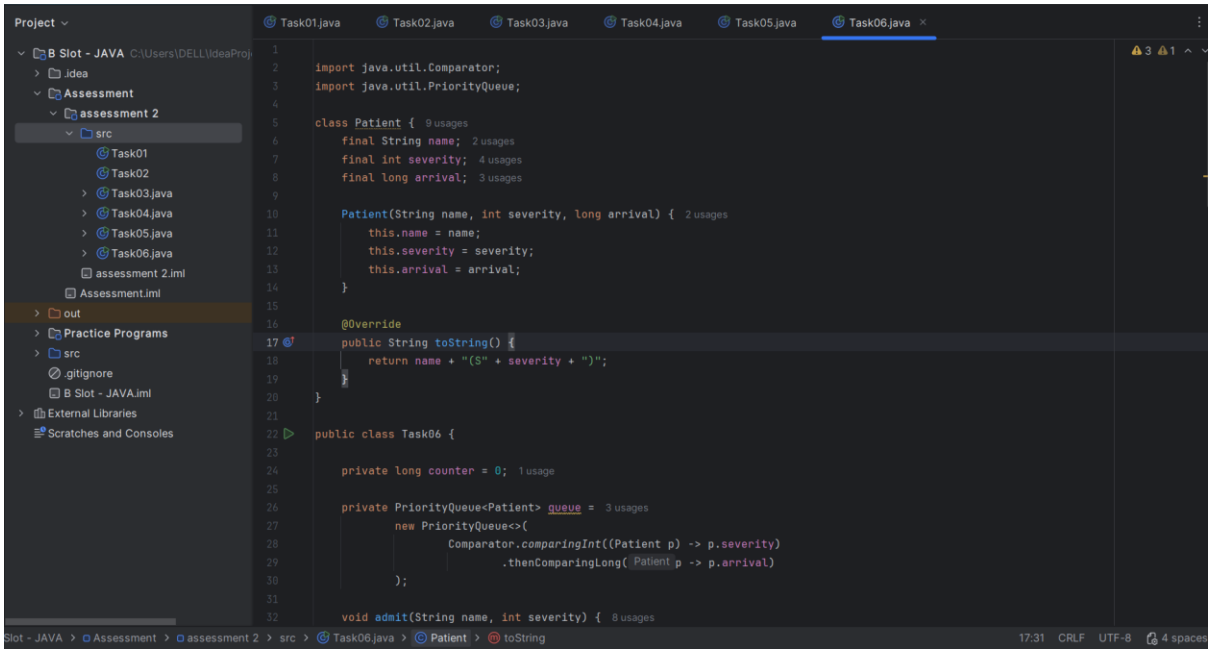


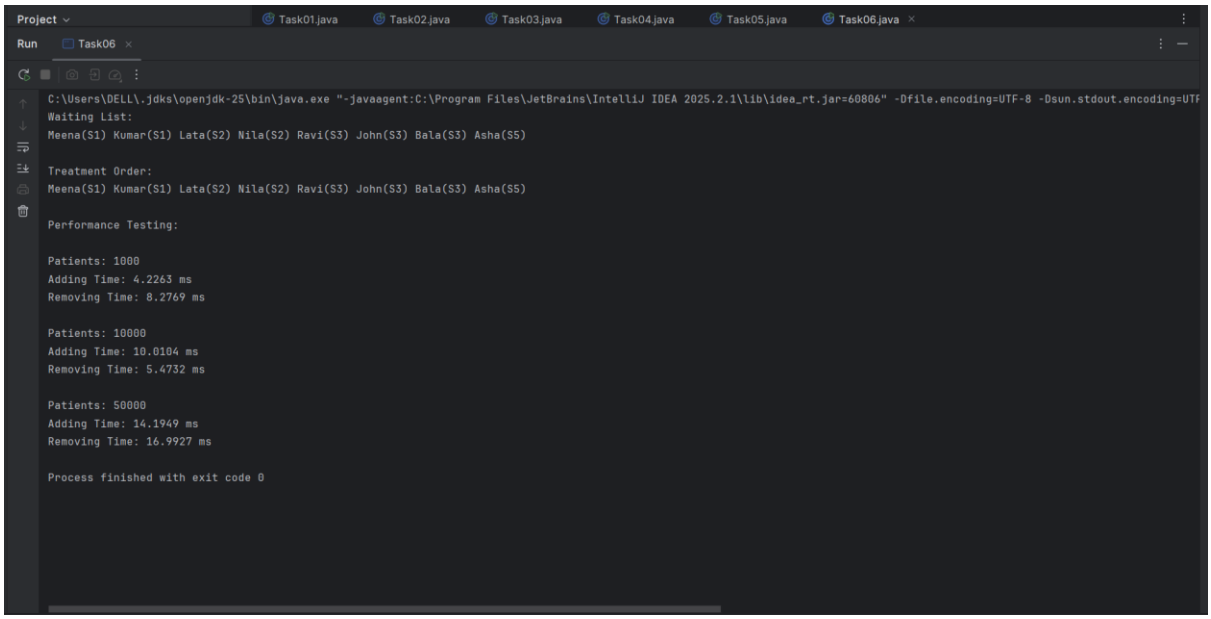
TASK 06

Q6–Priority, Order and Fairness in a Queue

Bloom's Level: K5– Evaluate Problem Context– Hospital Emergency Triage

Patients arriving at an emergency department are given a severity level from 1 (critical) to 5 (minor). The doctor on duty must always see the most severe patient next; among patients with the same severity, the one who arrived first must be seen first.





TASK 07

Q7–Diagnose,RedesignandValidate

Bloom'sLevel:K5–Evaluate

Review the evidence you generated while working on Q1–Q6. Select ONE solution that shows a significant limitation as the data size, usage pattern or requirement changes. You are not permitted simply to replace one collection with another and state that it is faster. You must understand the problem first and then develop an appropriate redesign.

A. Diagnose the Bottleneck

In the original railway ticket system, the + operator is used inside a loop to combine passenger details, PNR, coach, berth and fare.

Problems identified:

1. String is immutable, so every concatenation creates a new String.
2. Many intermediate String objects are created.
3. Execution time increases when the number of fields increases.
4. Extra memory is required for temporary objects.

Main bottleneck: Repeated String concatenation inside the loop.

B. Define the Design Objective

The redesigned system should:

1. Reduce unnecessary String object creation.
2. Improve ticket generation speed.
3. Reduce temporary memory usage.
4. Generate the same ticket output as the original program.

Measurable objective: Improve execution time when generating 10,000 tickets with 50 fields each, compared with the original String implementation.

C. Design the Revised Solution

Original design:

- Uses String.
- Joins fields using the + operator.
- Creates intermediate String objects during every concatenation.

Redesigned solution:

- Uses StringBuilder.

- Uses the append() method.
- Modifies the existing character buffer.
- Converts the final result into a String using toString().

Advantages:

- Faster text generation for repeated concatenation.
- Fewer intermediate String objects.
- Better performance for large tickets.

Trade-off: StringBuilder uses a mutable buffer that requires memory and is not thread-safe. A separate builder should be used for each ticket.

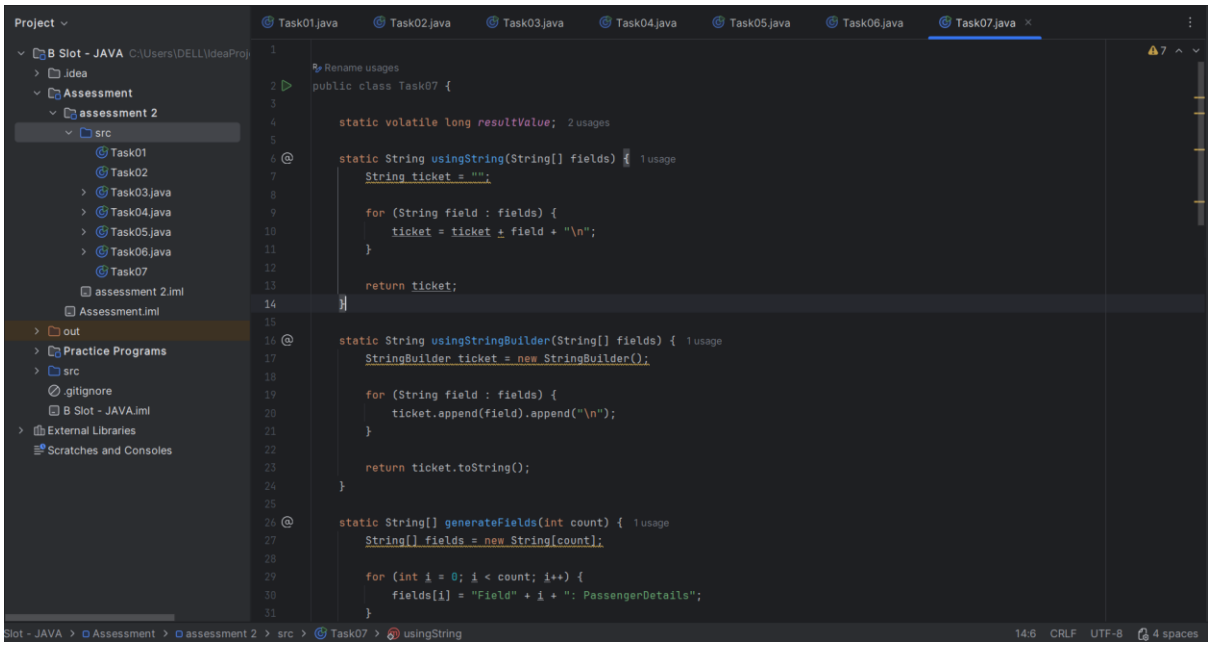
D. Predict Before Implementing

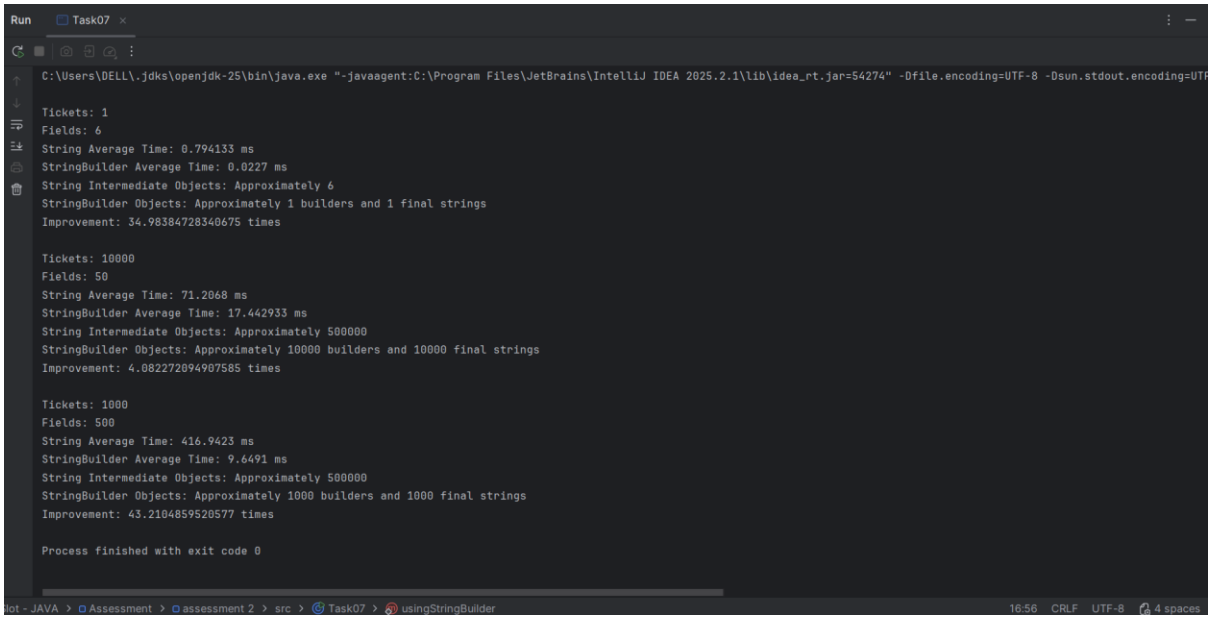
Before running the redesigned program, the expected results are:

- StringBuilder should take less execution time than String for large workloads.
- StringBuilder should create fewer intermediate objects.
- The improvement should be more noticeable when the number of fields increases.
- For a small ticket, the difference may be negligible.

E. Implement and Validate the Redesign

Complete Java program





F. Defend the Final Design – 2 Marks

1. Was the prediction supported?

The expected result is that StringBuilder performs better for large workloads because it avoids repeatedly creating new String objects. The actual timing results should be used to confirm this.

2. Improvement achieved

StringBuilder reduces repeated character copying and temporary object creation. The performance improvement becomes more important as the ticket size increases.

3. New costs and trade-offs

- StringBuilder requires a mutable buffer.
- It is not thread-safe.
- Each ticket needs its own builder.
- Small tickets may not show a significant performance difference.

4. When is the original solution preferable?

String is suitable for simple text joining with only a few fields and when readability is more important than repeated concatenation performance.

Final conclusion

The original railway ticket generator uses repeated String concatenation, which creates unnecessary intermediate objects and increases execution time for large workloads. The redesigned solution uses StringBuilder to improve performance and reduce temporary object creation. Therefore, StringBuilder is the better engineering choice for generating large numbers of railway e-tickets